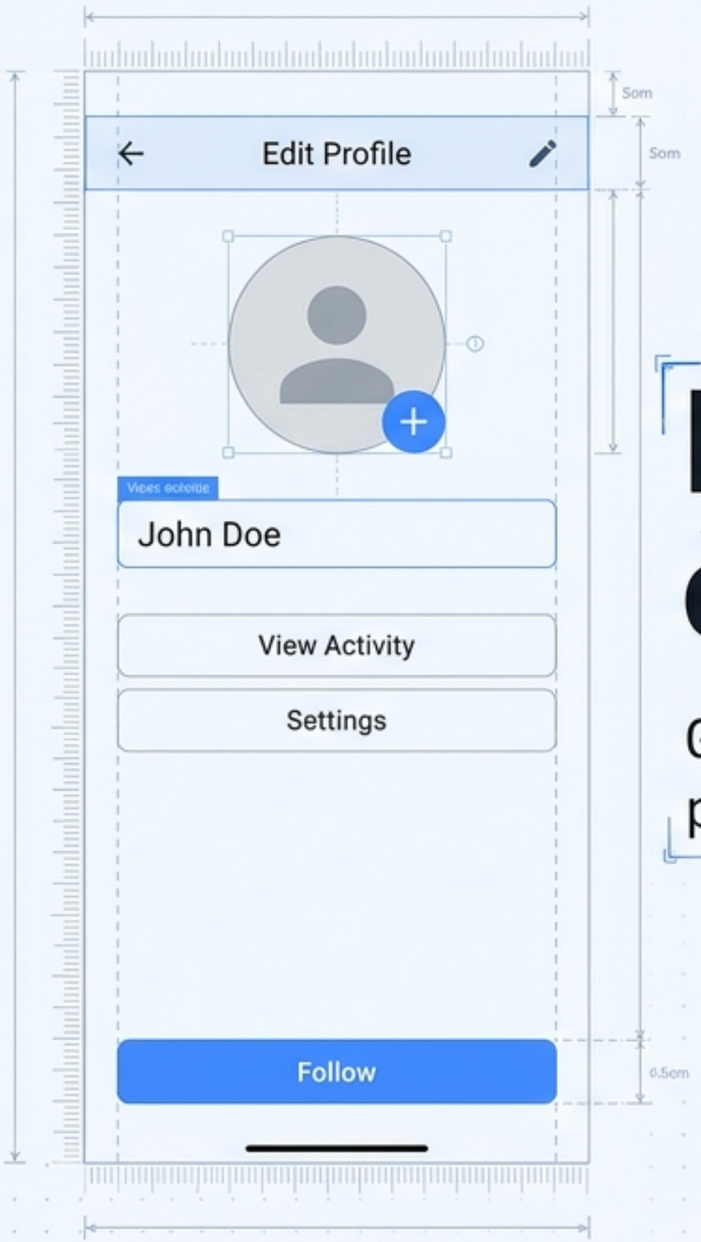


De la Maquette à la Compilation

Génération automatique de code GUI par IA avec YOLOv7 et Flutter.

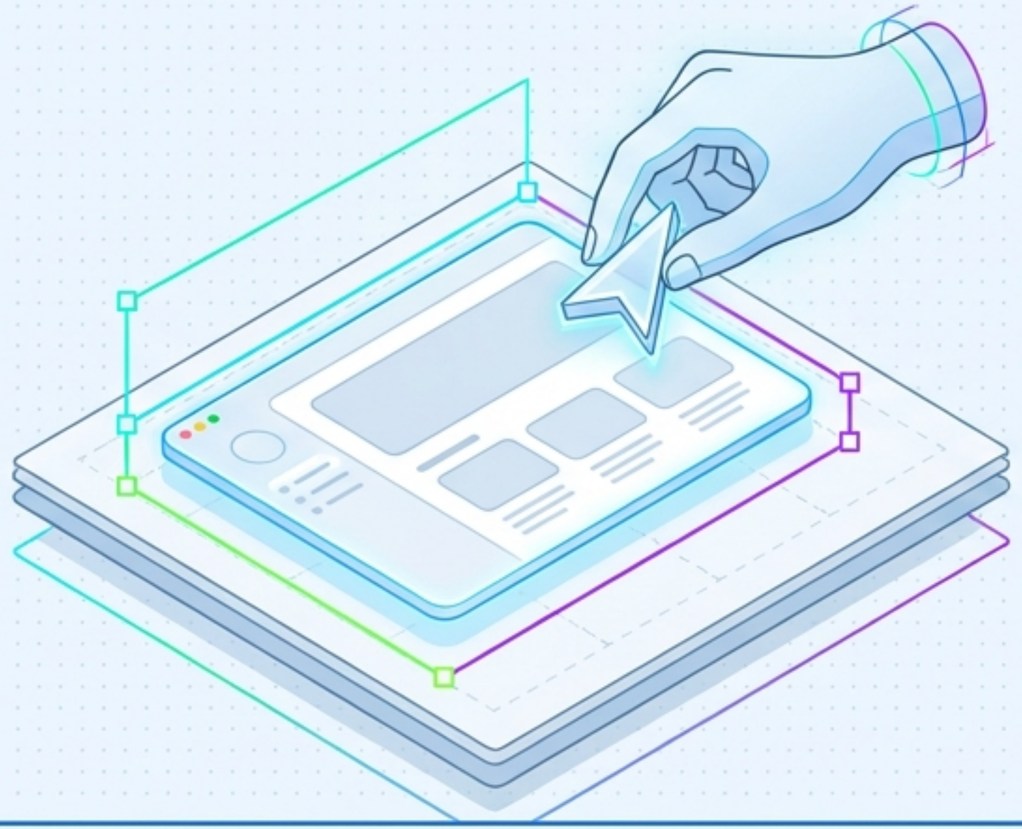
```dart
import 'package:flutter/material.dart';

class UserProfileScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      appBar: AppBar(
        title: const Text(
          'Edit Profile',
          style: TextStyle(color: Colors.black),
        backgroundColor: Colors.transparent,
        elevation: 0,
      ),
      body: SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.center,
```

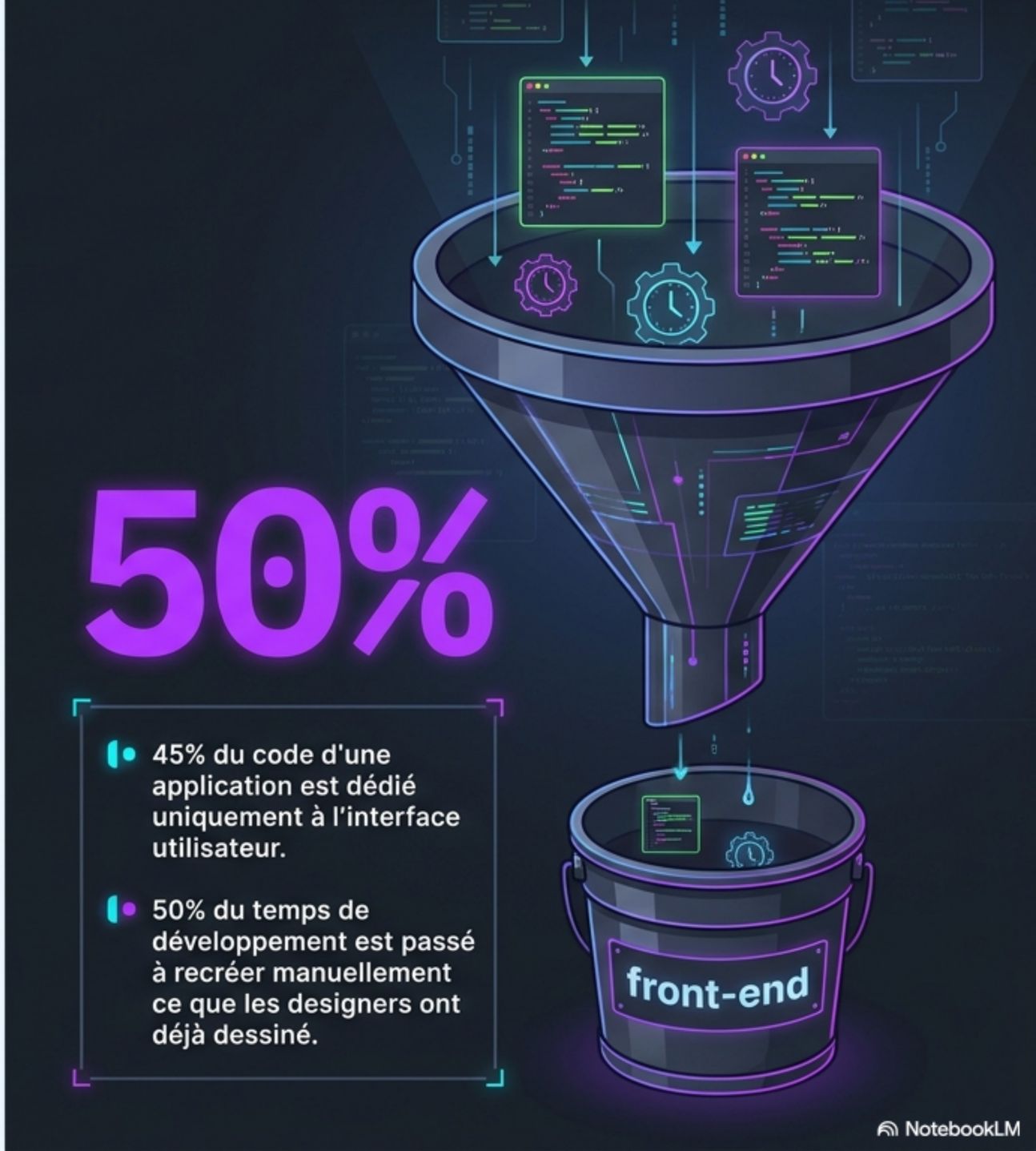
```dart
{
  "screen": "UserProfile",
  "elements": [
    {
      "type": "AppBar",
      "title": "Edit Profile"
      {
        "type": "ProfileImage",
        "source": "user_icon.png",
        "editable": true
      },
      {
        "type": "Text",
        "content": "John Doe",
        "style": "Heading1"
      },
      {
        "type": "Button",
        "label": "View Activity",
        "action": "view_activity"
      },
      {
        "type": "Button",
        "label": "Settings",
        "action": "open_settings"
      },
      {
        "type": "Button",
        "label": "Follow",
        "action": "follow_user",
        "primary": true
      }
  ]
}
```

NotebookLM

Le goulot d'étranglement du développement front-end



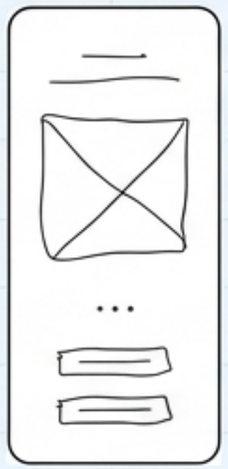
L'attente :
Focus sur la logique métier.



- 45% du code d'une application est dédié uniquement à l'interface utilisateur.
- 50% du temps de développement est passé à recréer manuellement ce que les designers ont déjà dessiné.

NotebookLM

Matrice de fidélité : Pourquoi cibler les maquettes ?



Croquis
(Hand-drawn)

- **Détail Visuel** : Faible (Brouillon).
- **Objectif** : Itération rapide.
- **Compatibilité IA** : Faible (Pas de standard universel).



Fil de fer
(Digital Wireframe)

- **Détail Visuel** : Moyen (Structurel).
- **Objectif** : Disposition globale.
- **Compatibilité IA** : Moyenne (Manque de style et de contenu).



Maquette Haute Fidélité
(Mockup)

- **Détail Visuel** : Élevé (Rendu final).
- **Objectif** : Design exact avant code.
- **Compatibilité IA** : Élevée (Prêt pour la traduction en code).

NotebookLM

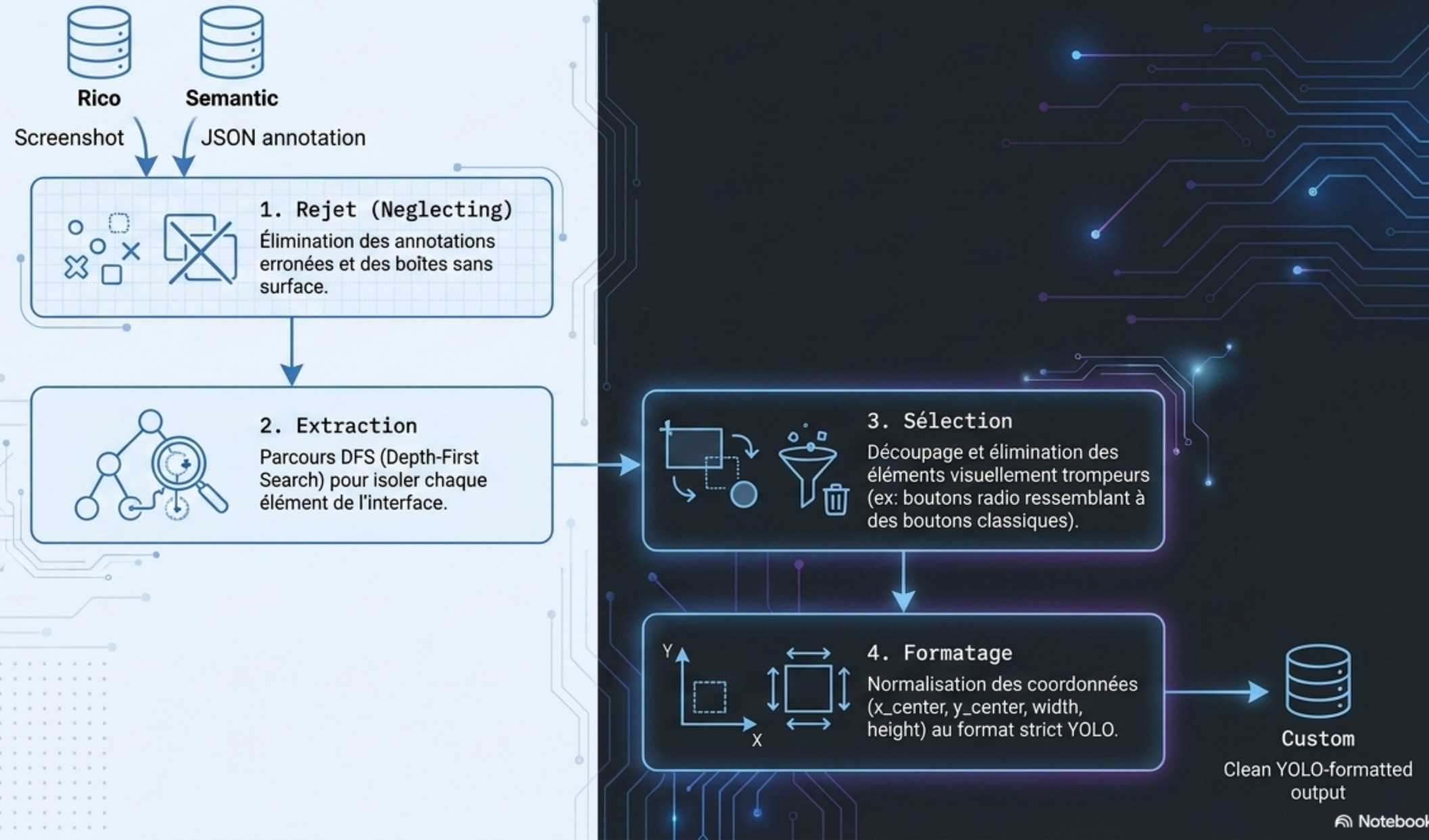
L’anatomie d’un algorithme : 5 phases vers l’automatisation



1. Préparation des données (Filtration)
2. Détection d’objets (YOLOv7)
3. Post-traitement (Extraction)
4. Hiérarchie de mise en page (Logique spatiale)
5. Génération de code (Flutter)

NotebookLM

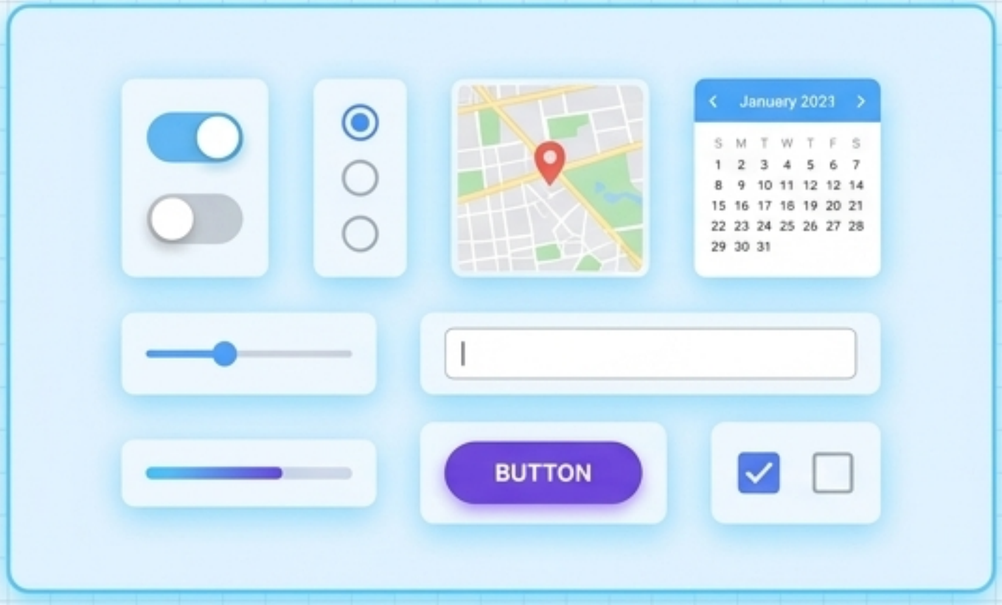
Phase 1 : Le système de filtration des données



NotebookLM

Phase 2 : Le problème d’échelle (Éléments Atomiques vs Conteneurs)

Éléments Atomiques

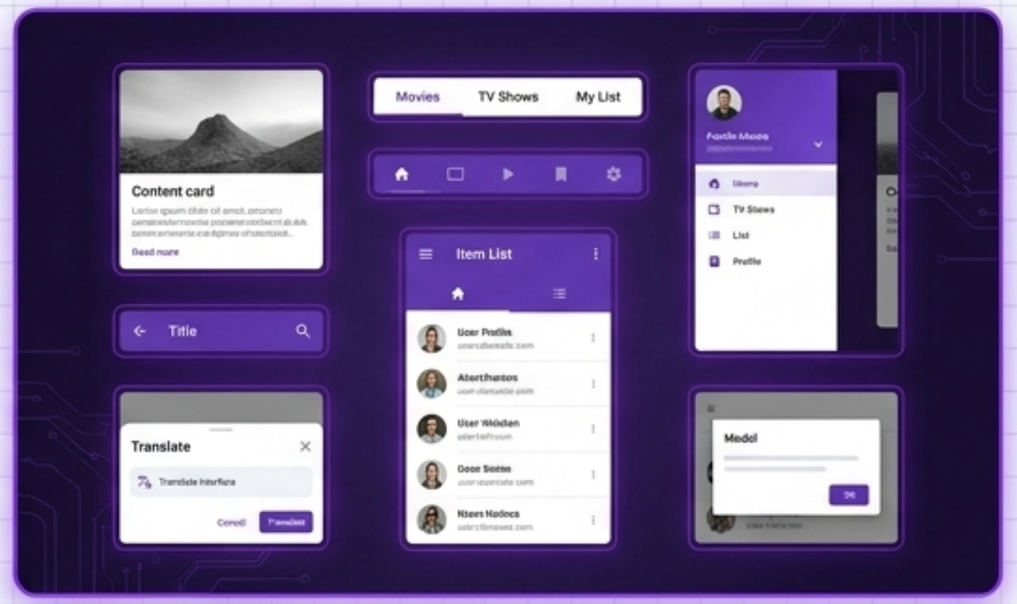


Définition : Les blocs de construction fondamentaux et indivisibles.

Exemples : Texte, Case à cocher, Interrupteur, Curseur.

Le Défi IA : Très petite taille, nécessite une haute précision locale.

Éléments Conteneurs



Définition : Les limites visuelles qui regroupent d’autres éléments.

Exemples : Barre d'outils, Tiroir (Drawer), Carte, Liste.

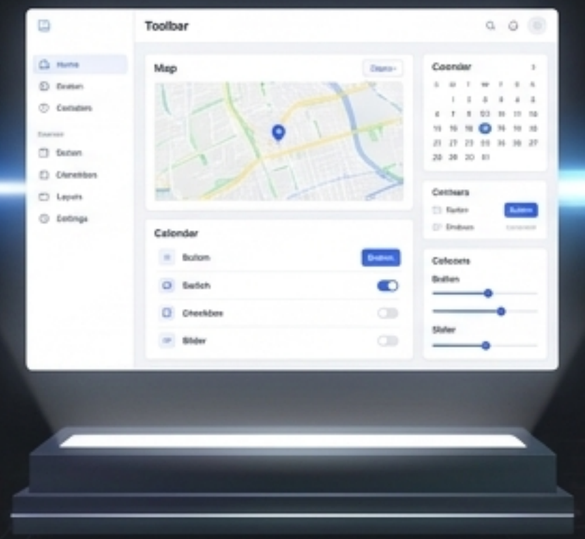
Le Défi IA : Grande échelle, risque d’interférence avec les objets inclus.

La Solution : Deux modèles YOLOv7 distincts pour éviter les interférences de classes et maximiser la précision.

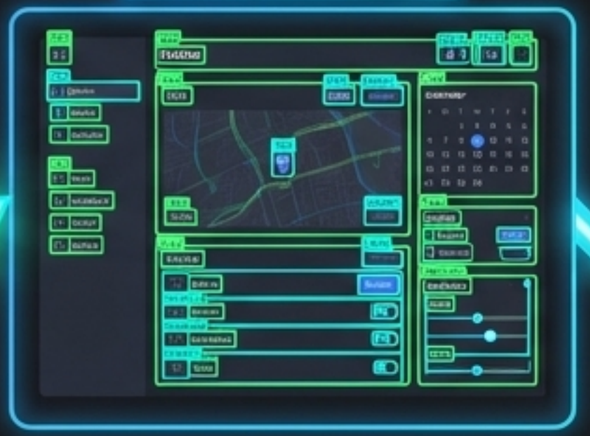
NotebookLM

Phase 2 : Le scanner à double lentille YOLOv7

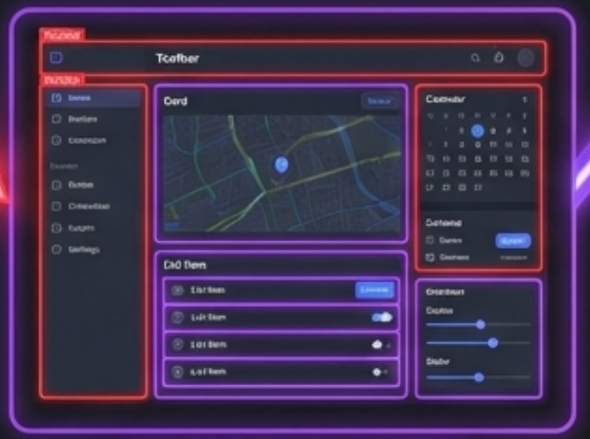
Prêt mochus de UI mockup arun les prévious slides (e.g.

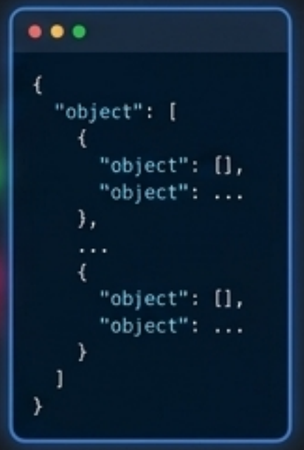


Modèle Atomique



Modèle Conteneur

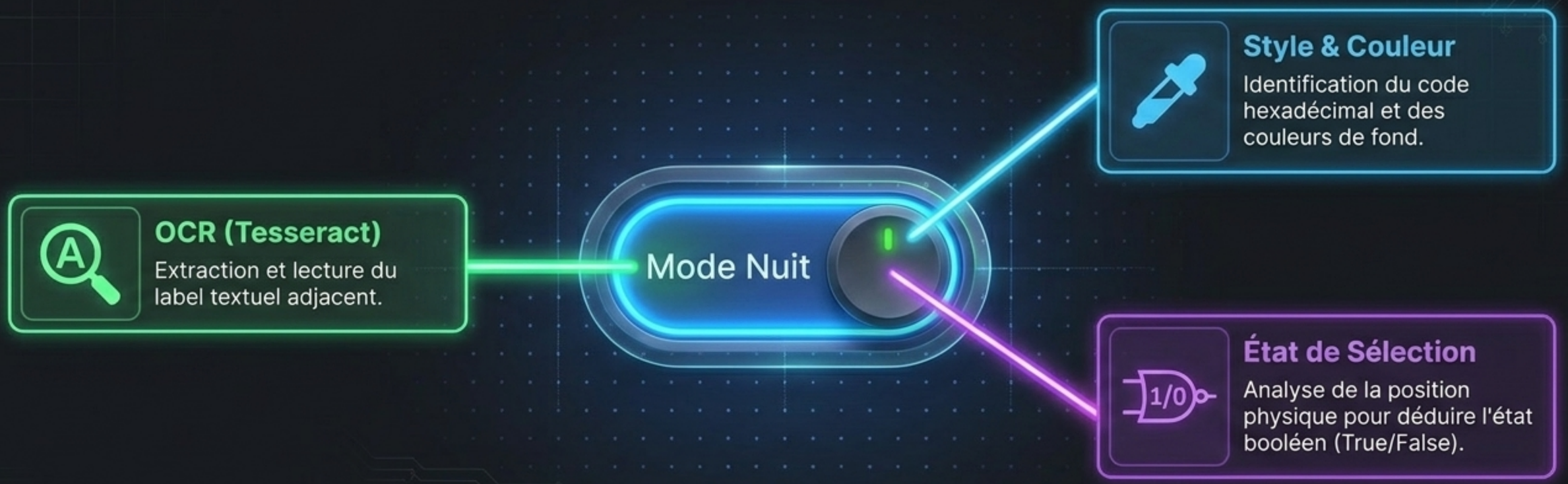




Les deux réseaux fusionnent leurs résultats dans une liste concaténée unique.

NotebookLM

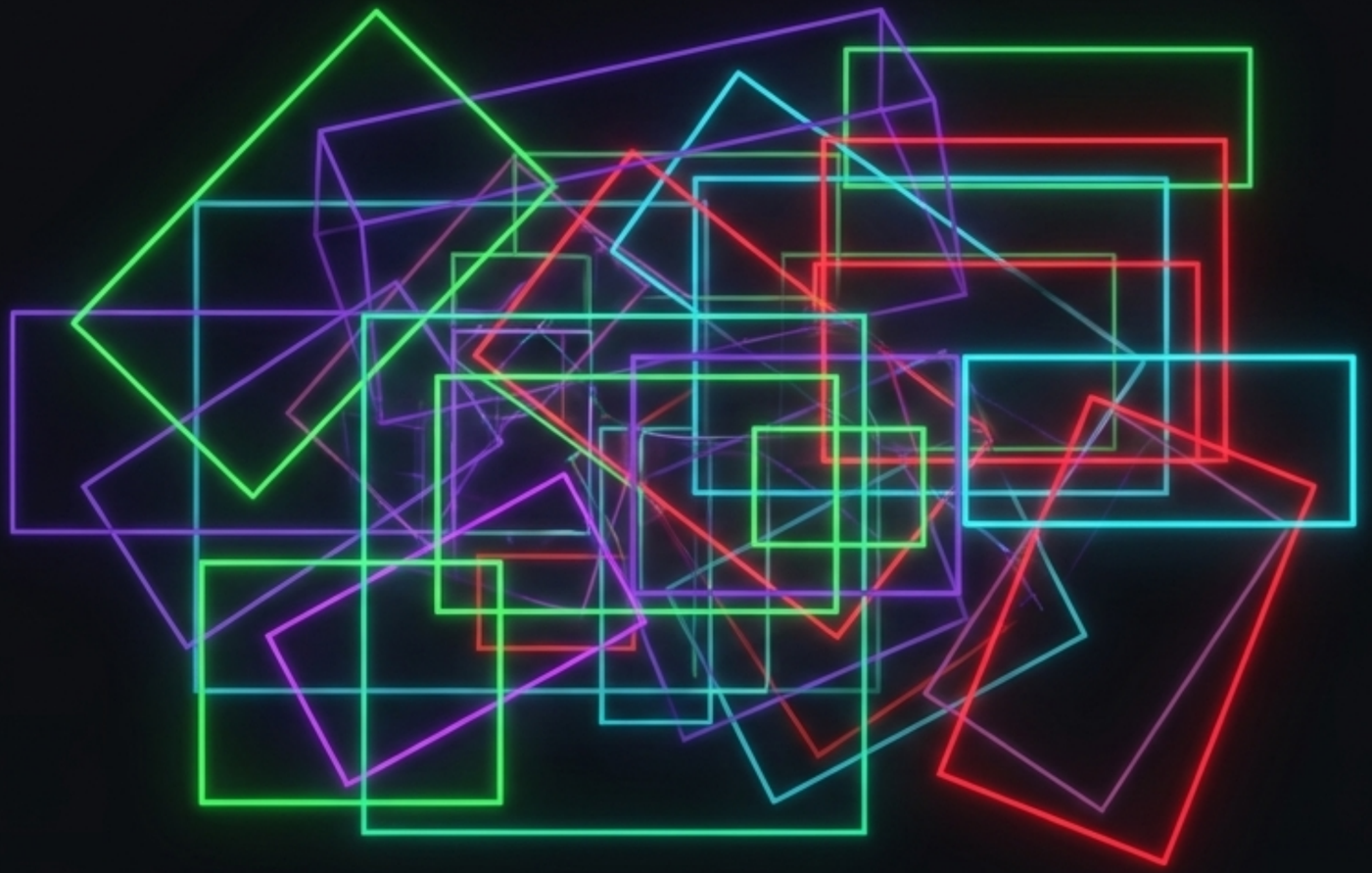
Phase 3 : Extraction des propriétés fonctionnelles



Génération d'un **objet de représentation UI** indépendant de la plateforme (dictionnaire JSON).

NotebookLM

L'Illusion de la Détection : Des Pixels sans Logique



L'erreur commune :

Si l'IA peut voir un bouton, elle peut coder l'application.

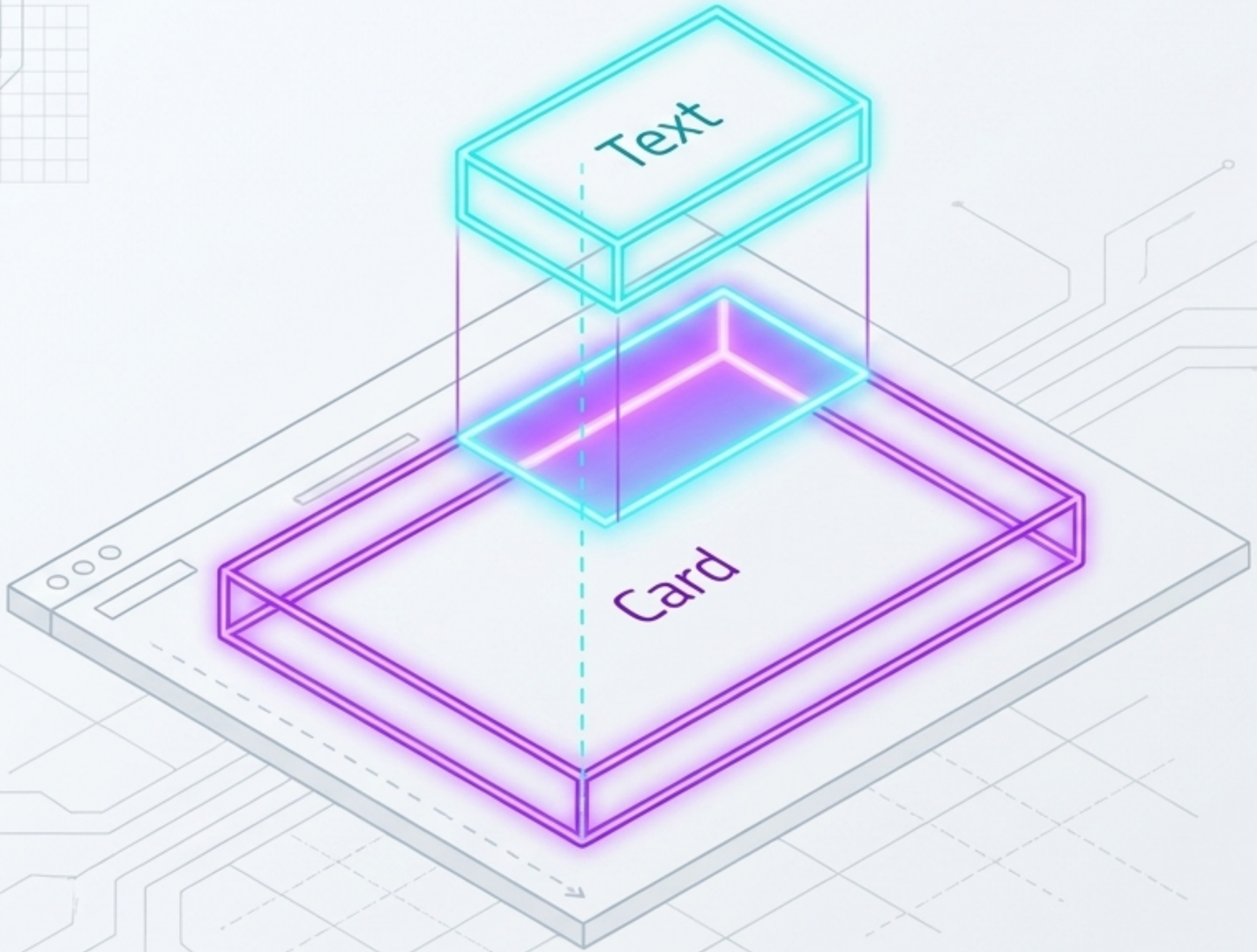
La réalité :

L'IA voit 32 éléments disparates. Sans compréhension spatiale, ce ne sont que des coordonnées brutes. Il manque la hiérarchie parent-enfant requise pour le code front-end (ex: colonnes et lignes Flutter).

La vision seule ne suffit pas. Il faut de la logique spatiale.

NotebookLM

Phase 4 : Construction de la hiérarchie spatiale

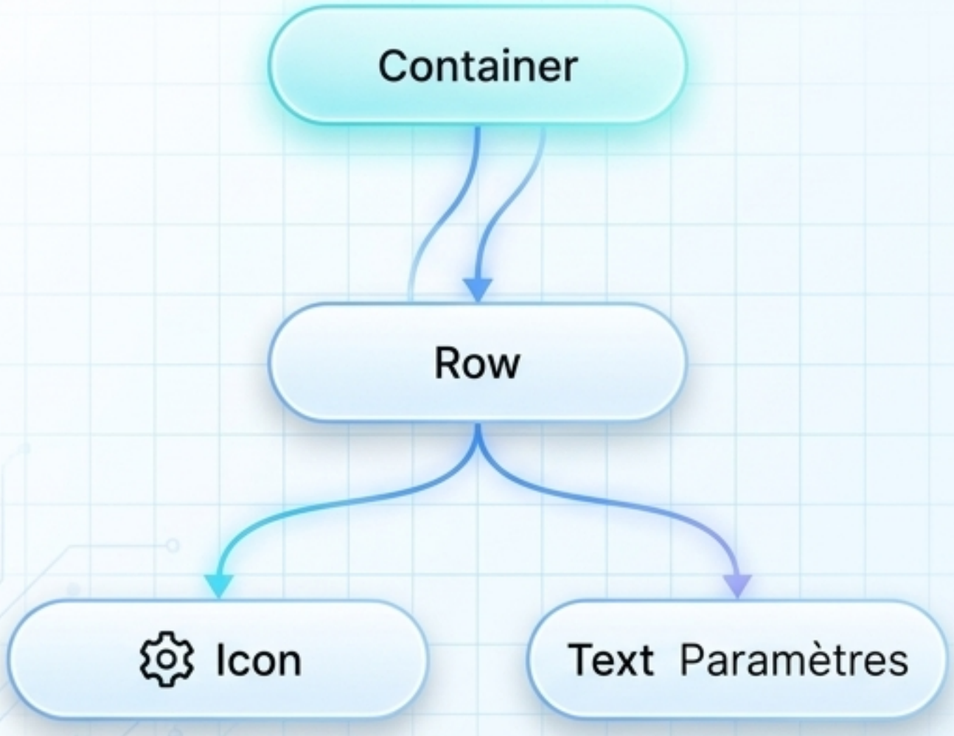


Le Test d'Intersection (Algorithme 6)

- La Règle d'Or : Si l'aire d'intersection dépasse 90%, l'élément atomique est classé atomique est classé comme "enfant" à l'intérieur du conteneur parent.
- Logique d'alignement : Des lignes verticales lient les éléments pour déterminer s'ils sont adjacents (alignement sur une même rangée horizontale).

NotebookLM

Phase 5 : Génération du code multiplateforme (Flutter)




```dart
Container(
  child: Row(
    children: [
      Icon(Icons.settings),
      Text('Paramètres'),
    ],
  ),
);
```

Key Insight : Utilisation de composants réutilisables (widgets d'état) plutôt que de primitives de plateforme, **garantissant** un **rendu identique** sur **iOS** et **Android** sans erreurs de compilation **Dart**.

NotebookLM

Tableau de bord des performances du modèle



Éléments Atomiques



Éléments Conteneurs

- Temps d'inférence : **45.6 ms à 85.0 ms** (ultra-rapide, en temps réel).
- Indice de similarité structurelle (SSIM) : **83.3%** de ressemblance visuelle entre la maquette originale et l'application générée.

Résultat : Le système garantit une haute précision à travers divers seuils de confiance sans biais d'apprentissage.

NotebookLM

Matrice de référence de l'industrie

Modèle	Technique	Utilisation OCR	Éléments détectés	Précision Globale
Pix2code	End-to-end	Non	15	77%
ReDraw	Hybride	Oui	Non spécifié	85%
UIED	Hybride	Oui	15	Non spécifié
Système Proposé (YOLOv7)	Détection d'objets	Oui	23	**88.2%**

NotebookLM

De la vision par ordinateur au déploiement en temps réel



- **Architecture à double modèle :** Élimine les interférences et résout le problème d'échelle.
- **Logique spatiale automatisée :** Convertit 23 types d'éléments sémantiques en arbres de widgets Flutter.
- **Impact :** Transforme une tâche manuelle laborieuse en une simple vérification de compilation, rendant aux développeurs 50% de leur temps pour se concentrer sur la logique métier.

L'interface utilisateur devient une compilation automatisée.

NotebookLM